

Multilayer Perceptrons, Part 2

Advanced Machine Learning

Sam Ogden

DRAFT

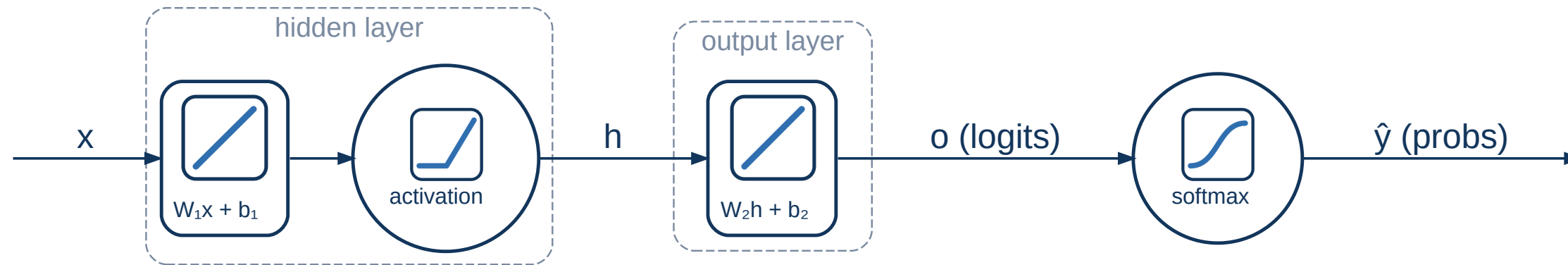
This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Recognize the output-layer gradient as a callback, not new material
- Derive how a gradient reaches a hidden layer it never directly touches
- Walk the full backward pass and one SGD update, on the example from last time

Recap: the forward pass

Where we left off



Same fruit classifier, same example: $x = [1.2, 0.2]$, true class banana.

stage	value
$h_{\text{pre}} = W_1x + b_1$	$[1.4, -0.8]$
$h = \text{ReLU}(h_{\text{pre}})$	$[1.4, 0]$
$o = W_2h + b_2$	$[0.7, -0.42, 0.28]$
$\hat{y} = \text{softmax}(o)$	$[0.5042, 0.1645, 0.3313]$
$L = -\log \hat{y}_{\text{banana}}$	1.8048

The Big Question

The loss only touches W_1 and b_1
indirectly — how does a gradient
reach them?

Gradients for the output layer, a callback

Nothing new here — you already did this

Same formula, new inputs

- From lecture 05, extended through the chain rule to a linear layer's own weights:

$$\frac{\partial L}{\partial o} = \hat{y} - y$$

- $o = W_2 h + b_2$ has the *exact same shape* as lecture 05's $o = Wx + b$ — just h standing in where x used to be:

$$\nabla W_2 = (\hat{y} - y) h^\top, \quad \nabla b_2 = \hat{y} - y$$

- No new derivation required — this is free progress from work already done

Gradients for the hidden layer

The actual new material

The problem, stated plainly

- W_1 and b_1 never appear in the loss formula directly — they only affect L by first changing h , which changes o , which changes $\hat{y}$
- The chain rule from lecture 05 had exactly **one** link between the parameter and the loss; here there are **three**: $W_1 \rightarrow h \rightarrow o \rightarrow \hat{y} \rightarrow L$

The Big Idea

The chain rule still works — you just follow one more link backward through the network.

Link 1: from the loss, back through W_2

- We already have $\partial L / \partial o = \hat{y} - y$
- $o = W_2 h + b_2$ is linear in h , so the chain rule passes the gradient straight back through W_2 :

$$\frac{\partial L}{\partial h} = (W_2)^\top (\hat{y} - y)$$

- Read $(W_2)^\top$ as **running the output layer backward** — instead of turning h into o , it turns an error in o into an error in h

Link 2: back through the activation function

- $h = \sigma(h_{\text{pre}})$, so one more chain-rule link is needed before reaching h_{pre} :

$$\frac{\partial L}{\partial h_{\text{pre}}} = \frac{\partial L}{\partial h} \odot \sigma'(h_{\text{pre}})$$

- $\odot$ means **elementwise** multiply — each hidden unit's incoming error gets scaled by that *same unit's own* activation derivative, independently of every other unit

ReLU's derivative

- For ReLU specifically, the derivative is especially simple:

$$\sigma'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

⚠ Important

If a unit's pre-activation was ≤ 0 , ReLU zeroed it going forward — and now its derivative is 0 too, so **no gradient passes through it at all**, no matter how large the incoming error was

Link 3: back through W_1

- Same move as lecture 05's very first gradient, just with h_{pre} 's error standing in for o 's:

$$\nabla W_1 = \frac{\partial L}{\partial h_{\text{pre}}} x^\top, \quad \nabla b_1 = \frac{\partial L}{\partial h_{\text{pre}}}$$

Breakdown: the same shape, three times over

Every one of these gradients has the exact same shape — “the error flowing into this layer, times what fed into it”:

layer	gradient	“what fed in”
output	$\nabla W_2 = (\hat{y} - y) h^\top$	h
hidden	$\nabla W_1 = \frac{\partial L}{\partial h_{\text{pre}}} x^\top$	x

i Note

Backprop isn’t a new kind of math for each layer — it’s the *same* local rule, applied once per layer, moving backward

Worked example: the backward pass

Setup, recalled from last lecture

$$x = [1.2, 0.2], \quad y = [0, 1, 0] \text{ (banana)}, \quad h_{\text{pre}} = [1.4, -0.8], \quad h = [1.4$$
$$o = [0.7, -0.42, 0.28], \quad \hat{y} = [0.5042, 0.1645, 0.3313]$$

Step 1: gradient at the output, $\partial L / \partial o$

$$\frac{\partial L}{\partial o} = \hat{y} - y = [0.5042, 0.1645 - 1, 0.3313] = [0.5042, -0.8355, 0.3313]$$

- Same shape as lab 04 — negative for the true class (banana), positive for the others

Step 2: gradient for W_2, b_2

Each row of ∇W_2 is that class's Step 1 value, scaled by $h = [1.4, 0]$ — same “one dot product per row” pattern as lab 04's update step:

$$\nabla W_{2,\text{apple}} = 0.5042 \times [1.4, 0] = [0.7059, 0], \quad \nabla W_{2,\text{banana}} = -0.8355$$

$$\nabla b_2 = [0.5042, -0.8355, 0.3313]$$

- Every row's second entry is 0 — because $h_2 = 0$, unit 2 contributed nothing forward, so it gets no credit or blame backward either

Step 3: gradient at the hidden layer, $\partial L / \partial h$

$$\frac{\partial L}{\partial h} = (W_2)^\top (\hat{y} - y)$$

$$\frac{\partial L}{\partial h_1} = (0.5)(0.5042) + (-0.3)(-0.8355) + (0.2)(0.3313) = 0.5690$$

$$\frac{\partial L}{\partial h_2} = (1)(0.5042) + (2)(-0.8355) + (-1)(0.3313) = -1.4981$$

$$\frac{\partial L}{\partial h} = [0.5690, -1.4981]$$

- Unit 2 *would* get a large error signal (-1.4981) — but keep going, ReLU isn't done with it yet

Step 4: gradient at the pre-activation, $\partial L / \partial h_{\text{pre}}$

$$\frac{\partial L}{\partial h_{\text{pre}}} = \frac{\partial L}{\partial h} \odot \sigma'(h_{\text{pre}}), \quad \sigma'(h_{\text{pre}}) = [1, 0] \quad (\text{since } h_{\text{pre}} = [1.4, -0.8])$$

$$\frac{\partial L}{\partial h_{\text{pre}}} = [0.5690 \times 1, -1.4981 \times 0] = [0.5690, 0]$$

Warning

Unit 2's error signal got **erased** by ReLU's derivative — that large -1.4981 from Step 3 never reaches W_1 's second row at all

Step 5: gradient for W_1, b_1

$$\nabla W_1 = \frac{\partial L}{\partial h_{\text{pre}}} x^\top$$

$$\nabla W_{1,1} = 0.5690 \times [1.2, 0.2] = [0.6828, 0.1138], \quad \nabla W_{1,2} = 0 \times [1.2,$$

$$\nabla b_1 = [0.5690, 0]$$

- Unit 1's row gets a real update; unit 2's row gets **exactly zero** — this example teaches unit 2 nothing

The update, and what backprop actually is

Step 6: apply SGD to all four parameter tensors

Same update rule as always, $\theta \leftarrow \theta - \eta \nabla \theta$, with $\eta = 0.5$, applied to every parameter this network has:

Hidden layer

$$W_{1,\text{new}} = \begin{bmatrix} 0.6586, & 0.9431 \\ -1, & 2 \end{bmatrix}$$

$$b_{1,\text{new}} = [-0.2845, 0]$$

Output layer

$$W_{2,\text{new}} = \begin{bmatrix} 0.1471, & 1 \\ 0.2848, & 2 \\ -0.0319, & -1 \end{bmatrix}$$

$$b_{2,\text{new}} = [-0.2521, 0.4177, -0.1656]$$

 Note

W_1 's second row is unchanged — $[-1, 2]$ before and after. A zero gradient means zero update, no matter what η is

Step 7: verify — did that step help?

Run the same forward pass again, with the updated parameters:

$$\mathbf{o}_{\text{new}} = [-0.15, 0.6155, -0.1878], \quad \hat{\mathbf{y}}_{\text{new}} = [0.2431, 0.5228, 0.2341]$$

$$L_{\text{new}} = -\log(0.5228) \approx 0.6486$$

- Loss went from **1.8048** → **0.6486** — a bigger drop than lab 04's single-layer example
- And banana now **wins**: 0.5228 is the highest probability, up from third place. One step, two layers, already flipped the decision

What backpropagation actually is

Backpropagation is the chain rule, applied once per layer, moving backward from the loss.

- Nothing here was a special “neural network trick” — every step was ordinary chain-rule calculus, applied to one link in the chain at a time
- This generalizes: a network with 10 hidden layers uses the exact same idea, just with more links — compute the error at the output, then repeatedly ask “what was this layer’s input, and how did it get to me?”

Summary

The full recipe

Forward

$$x \rightarrow W_1x + b_1 \rightarrow \text{ReLU} \rightarrow h \rightarrow W_2h + b_2 \rightarrow \text{softmax} \rightarrow \hat{y} \rightarrow L$$

Backward

$$\frac{\partial L}{\partial o} \rightarrow \frac{\partial L}{\partial h} \rightarrow \frac{\partial L}{\partial h_{\text{pre}}} \rightarrow \nabla W_1, \nabla b_1$$

Update

$\theta \leftarrow \theta - \eta \nabla \theta$, for every parameter tensor,
same rule as always

Note

This recipe — forward, then chain-rule backward one layer at a time, then update — is the template for training *any* feedforward network, not just this two-layer one

Tip

In lab: walk this exact example's full forward and backward pass **by hand**, including the dead-unit gradient